

Exercises

Exercise 7: Reproducible reports with R markdown

Read Chapter 8 to help you complete the questions in this exercise.

Before the session. This exercise needs the `rmarkdown` and `knitr` packages, and a LaTeX installation so that you can make PDF documents. Work through **Setting up R markdown**, under the Setup menu above, before you come to the session. It takes about fifteen minutes and most of that is a download running by itself, but please don't leave it until the morning of the session. If you hit a real problem we'll help you at the start of the session, but we can't get everybody set up and still have time to teach anything.

Up to now everything you've written has been code, for your own use. This exercise is about producing a *document* for somebody else to read, in which your writing, your code and your results all live in the same R markdown file. Because they're in the same file they can't drift apart, which is what we mean by a reproducible report. Change the data, press one button, and every number, figure and table in the output file updates itself.

It's also the exercise where the rest of the course comes together. You won't be learning much new R here. Instead you'll be taking the things you already know how to do, importing data, checking it for values that can't be right, transforming a variable, drawing a plot and summarising into a table, and setting them out in the order a reader needs them: what the data are, what you did to them and why, and what you found. That order is the point. A report is not a pile of results, it's an account of a piece of work.

You'll build **one document** across the whole exercise, adding something to it at each question, so don't start a new file each time. There's a lot of new vocabulary here, so take it slowly and knit often. Knitting after every small change is the quickest way to see what each thing does, and the quickest way to find out which change broke something.

1. Open the RStudio Project you have been using for this course. Create a new R markdown document by going to **File -> New File -> R Markdown...**

The first time you do this, RStudio may tell you it needs to install some packages. Let it, and wait for it to finish.

In the dialogue box that appears, type **Cardiac study report** in the Title box, put your name in the Author box, select **PDF** on the left, and click OK. RStudio opens a new document that's already full of example content about cars. That's normal, and we'll delete it in a moment.

Before you change anything at all, save the file into your Project directory as **cardiac_report.Rmd** (**File -> Save As...**), then click the **Knit** button at the top of the editor. You can also press **Ctrl+Shift+K**, or **Cmd+Shift+K** on a Mac.

Do this before you understand any of it. If a PDF appears then your setup is working, and you can be confident that anything which goes wrong later is something you can fix.

2. Look at the PDF that appeared alongside the R markdown file that produced it. An R markdown file has only three kinds of content, and everything else is a variation on them:

- the **YAML header** at the very top, between the two lines of `---`, which controls the document as a whole
- **text**, written in a simple markup language called markdown
- **code chunks**, which look like this:

```
```{r}
mean(cardiac$age)
```
```

Find one of each in your file. Then delete everything below the first code chunk, the one called `setup`, leaving the YAML header and that chunk in place. Knit again to check you have an empty but working document. This is your starting point.

3. Now take control of the YAML header. At the moment it has a title, an author and a date, and an output line saying `pdf_document`. Replace that line with the four shown below.

Two warnings, because this is where people lose ten minutes. Indentation in YAML matters, so copy the spacing exactly. And use spaces, never tabs.

```
---
title: "Cardiac study report"
author: "Your Name"
date: "20 August 2026"
output:
  pdf_document:
    toc: true
    number_sections: true
---
```

Add `toc: true` first and knit. Then add `number_sections: true` and knit again. Adding them one at a time, and knitting after each, is how you see what each one actually does.

4. Every report starts by telling the reader what the data are, so that's where you'll start too. Underneath the setup chunk, write a short introduction to the cardiac study using markdown formatting only, with no code yet. Include:

- a heading, written as `# My heading`
- a second, smaller heading below it, written as `## My smaller heading`
- a sentence with a word in **bold**, written as `**bold**`, and a word in *italics*, written as `*italics*`
- a bulleted list of four of the variables in the dataset, each line starting with a dash and a space

That's enough to be going on with. If you want to add a link, a numbered list, a quotation or a table, Section 8.5.2 of the book has the syntax for all of them, and there's a summary on the R markdown cheat sheet under `Help -> Cheat Sheets`.

Knit, and put the R markdown file and the PDF side by side. Notice that the headings you wrote have appeared in the table of contents on their own, because of the `toc: true` you added in Q3.

5. Next comes the data. Insert a new code chunk using the **Insert** button at the top right of the editor and choosing R, or with the keyboard shortcut Ctrl+Alt+I (Cmd+Option+I on a Mac). An empty chunk appears. If you'd rather, there's nothing magic about the button: you can simply type ````{r}` on a line of its own and ````` on another line below it, and anything you put between them is a chunk.

Give the chunk a name by typing it straight after the `r`, so the first line reads ````{r import}`. Naming chunks costs nothing and makes them far easier to find when something goes wrong.

Inside the chunk, import `cardiacdata.txt` exactly as you did in **Exercise 3, Q5**: with the `read.table()` function, writing out the `header`, `sep` and `stringsAsFactors` arguments, and assigning the result to `cardiac`. In the same chunk, add the two `factor()` lines from **Exercise 3, Q7** that create `Fsex` and `Fsmoking`, since you'll want them later.

Run the chunk in the editor with the small green arrow at its top right, then knit the whole document. Notice that knitting starts a completely fresh R session: anything you made earlier in the console doesn't exist when the document knits, which is exactly what makes it reproducible. It also means the chunk that imports the data has to come before any chunk that uses it.

6. You have the data in, so the next thing a reader needs to know is whether you can trust it, and what you did about the bits you couldn't. This is the question that most resembles the writing you'll be doing later in the course, because it asks you not just to do something to the data but to explain why.

Add a new heading called **## Data validation** below the previous chunk, and underneath it a chunk named `validation` holding the three lines from **Exercise 4, Q1** that set the impossible `bmi`, `triglyceride` and `hdlchol` values to `NA`. Remember that your report imports the raw file from scratch every time it knits, so if the cleaning isn't in the document then it hasn't happened.

Then write a short paragraph beneath the chunk, two or three sentences, saying which variables you changed, how many values in each, and why you set them to `NA` rather than deleting those patients or correcting the numbers yourself. Type the counts in for now. You'll come back to them in Q9, and there's a wrinkle waiting for you there.

You'll remember from **Exercise 5, Q9** that `alcohol` needed dealing with as well. A small number of patients drink a great deal more than the rest, which leaves the distribution with a long tail to the right, and you took the square root to pull that tail in. That's a data preparation step exactly like the cleaning above, so it belongs in this section too. Add a chunk named `alcohol-transform` that creates the square root of `alcohol` and draws the histograms of `alcohol` and `sqrt(alcohol)` side by side. Give it a caption with `fig.cap`, and write a sentence or two underneath saying why you did it.

7. With the data in and tidied up, you can start showing the reader what's in it. Insert another chunk, name it `chol-plot`, and use it to redraw the boxplot of HDL cholesterol by smoking status from **Exercise 5, Q11**. You already have `Fsmoking` from your import chunk, so this is just the `boxplot()` call.

Now give the figure a caption. Chunk options go inside the curly braces, after the chunk name, separated by commas. The first line of your chunk should read:

```
```{r chol-plot, fig.cap = 'HDL cholesterol by smoking status.'}
```
```

Knit and find your caption underneath the plot. Then add `fig.width = 4` to the same line, knit again, and see what happens to the size of the text relative to the plot.

There are a great many other chunk options for figures, controlling height, alignment, resolution and so on. You don't need them today. Chapter 8 of the book lists the ones worth knowing.

8. A plot shows a reader the shape of something, but they'll often want the numbers as well, so add a table next to go with your figure. If you simply print a dataframe in a chunk you get the console output, monospaced and ugly. The `kable()` function from the `knitr` package formats it as a proper table instead.

Insert a chunk, name it `summary-table`, and start by recreating the `aggregate()` summary from **Exercise 4, Q4**: the mean of `age`, `systolic` and `tchol` for each smoking category. Assign it to an object called `smoke_summary`.

Knit and look at how ugly it is. Now put `knitr::kable(smoke_summary)` on the next line, knit again, and compare. Finally add `digits = 1` and a `caption` to the `kable()` call to tidy it up.

9. Your report is now more or less complete, so the rest of the exercise is about tidying it up. Start with the paragraph you wrote in Q6. You typed those numbers in yourself, and numbers typed into your writing go stale. If the data changes, or you correct something later, your writing quietly stops matching your results and nothing tells you.

R markdown fixes this with **inline code**. In the middle of an ordinary sentence you write a backtick, then the letter `r`, then some R code, then a closing backtick, and when you knit, the answer appears instead. Like this:

```
The study included `r nrow(cardiac)` patients.
```

which comes out as “The study included 163 patients.”

Go back to your data validation paragraph and replace the numbers you typed with inline code. Then add a sentence to the same section giving the number of patients and their mean age in the same way. Use `round()` on the mean so you don't get six decimal places.

Then test that it really works. Temporarily change your import chunk so it keeps only the first 100 patients, by adding this line at the end of the chunk:

```
cardiac <- head(cardiac, 100)
```

Knit, and watch your sentences update themselves. Then delete that line and knit again.

10. Now think about who's going to read your report, because it decides how much of your working they should see. This one is a technical document, written for people who know R, so showing the code is exactly right and everything you've done so far is fine as it is. But you'll often write for readers who don't know R at all, a clinical team, a service manager, a patient group, and for them a wall of code is just something to scroll past on the way to the results.

You control this chunk by chunk with the `echo` option, in the same curly braces as the caption you added in Q7. Add `echo = FALSE` to your `chol-plot` chunk and knit. The plot is still there, but the code that drew it is not. Take it off again afterwards, since we want the code visible in this report.

Two other things arrive uninvited in reports, and they're worth knowing about now so they don't surprise you later. **Messages** are the chatty output R produces when it's doing something in the background, most often when you load a package with `library()`, and they can run to several lines. **Warnings** are R telling you something might not be right, and they look far more alarming to a reader than they usually deserve to. Both appear in your document in the middle of your writing. You turn them off the same way, with

`message = FALSE` and `warning = FALSE`, either on a single chunk or on all of them at once by putting them inside the `knitr::opts_chunk$set()` call in your `setup` chunk.

Be careful with `warning = FALSE` though. You're hiding warnings from your reader, not from yourself, so keep an eye on the ones appearing in the console while you work, because some of them matter.

11. Not every figure in a report is one that R drew. You might need a diagram, a photograph, a map, or a figure somebody else made. Add the plot you exported in **Exercise 6, Q9**, `ex6_admissions.png`, to your report.

The simplest way needs no chunk at all. Type this straight into your text, changing the path to wherever your file actually is:

```
![Alcohol-related hospital admissions by council area.](output/ex6_admissions.png)
```

An exclamation mark, then the caption in square brackets, then the file path in round brackets. Knit and check the figure appears.

If you no longer have the file, download it [here](#) and save it into your `output` directory, the one you created in **Exercise 1, Q12**.

One thing that catches everybody out: the path is relative to where the `.Rmd` file is, not to your working directory. If the image doesn't appear, the path is almost always the reason.

12. If you have time, here's one of the nicer things about R markdown: the same R markdown file will give you a completely different kind of document without you rewriting a word of it. In your YAML header, change `pdf_document` to `html_document` and knit. You'll get a web page instead, built from exactly the same text, code and figures. While you're there, add `toc_float: true` on its own line underneath `toc: true` and knit again to see the contents float alongside the page as you scroll. That option only works in HTML, so change the header back to `pdf_document` and delete the `toc_float` line before you finish.

Two things worth knowing about, but not now

The visual editor. RStudio can show an R markdown document formatted as you type, more like a word processor, hiding the markdown syntax. You turn it on with the small compass icon at the top right of the editor. It's genuinely useful, especially for tables and links. We've deliberately left it until the end so that you learn what the markup actually is first, because when something goes wrong it's the markup you have to fix.

Quarto. Quarto is the successor to R markdown. It does the same job, works with languages other than R, and its syntax is so close that nearly everything in this exercise transfers unchanged. If you carry on using R after this course you'll meet it. Start at quarto.org.

End of Exercise 7